

Lab Task Threads

Lab Task 1: Multi-Taxi Service (Handling More Customers)

Objective:

- Implement a **multi-threaded taxi service** where multiple taxis **pick up multiple customers simultaneously**.
- Ensure **each taxi operates independently** but the main program **waits for all taxis to finish**.

Requirements:

1. Ask the **user to enter the number of customers** (e.g., 5 to 10).
 2. Generate **customer IDs dynamically** (e.g., 101, 102, 103...).
 3. Create **one thread per taxi**, each picking up one customer.
 4. Use **pthread_join** to wait for all taxis to complete.
-

Lab Task 2: Multi-Restaurant Food Delivery (Handling More Orders)

Objective:

- Implement a **multi-threaded food delivery system** where multiple drivers **deliver multiple orders**.
- Each driver should handle **two orders at the same time**.

Requirements:

1. Ask the **user to enter the number of deliveries** (e.g., 6 to 10 orders).
 2. Use a **struct (Order)** to store **two customer names and two order IDs per driver**.
 3. Create **one thread per driver**, each delivering **two orders** before completing.
 4. Use **pthread_join** to ensure all deliveries finish before printing **"All deliveries are completed!"**.
-

Lab Task 3: Multi-Threaded Calculation System

Objective:

- Implement a **multi-threaded system** where multiple users **perform mathematical calculations** using threads.
- Each user (thread) provides **two numbers** and selects an **operation (addition, subtraction, multiplication, or division)**.

Requirements:

1. Ask the **user to enter the number of calculations** to be performed (e.g., 5 to 10).
2. Each **thread takes two numbers and an operation** as input and computes the result.
3. Use **pthread_join** to ensure all calculations complete before displaying "**All calculations are completed!**".
4. Print the **result of each calculation** in a structured format.

THE END
